DB 구현

공공데이터 융합 풀스택 개발자 양성과정

2조 - 강현욱 김선길 박한결 박영준

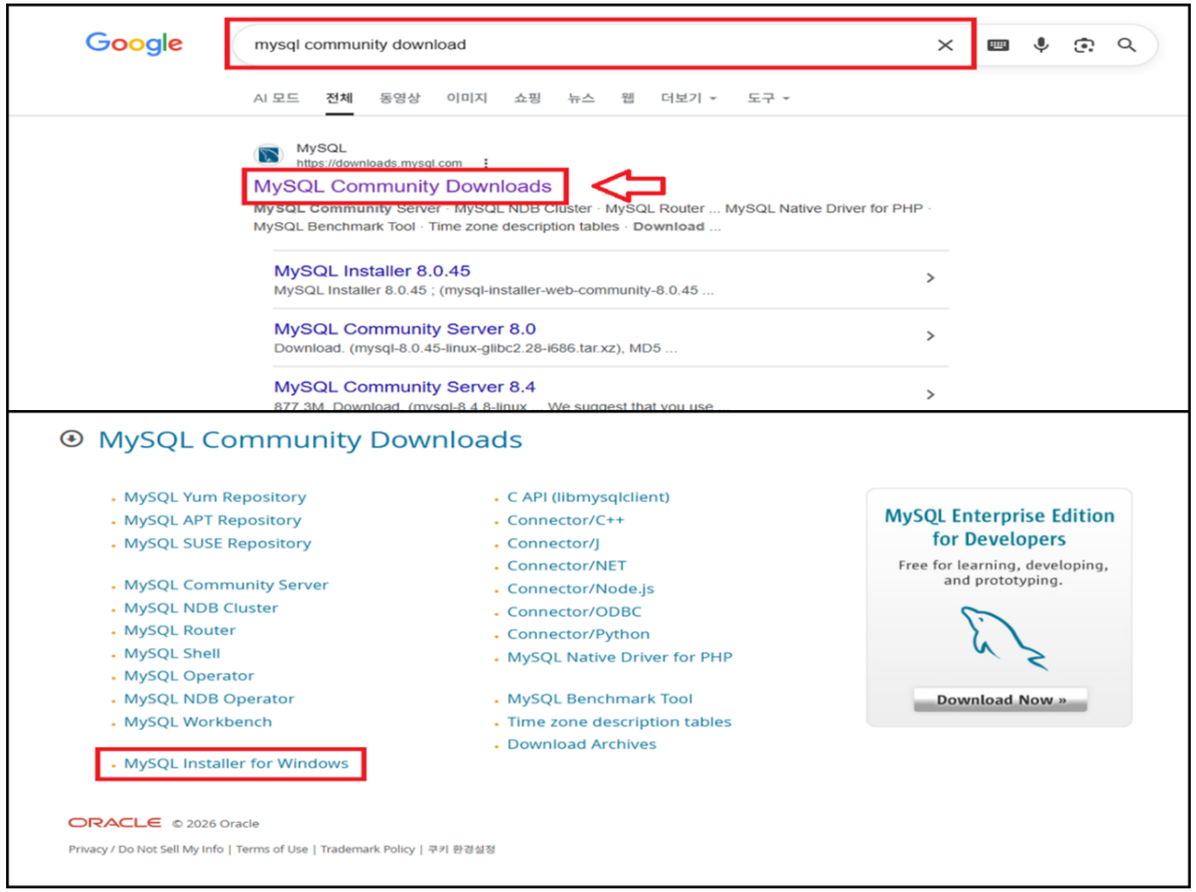
INDEX

01 DB 설치

02 DB 계정생성

03 DB ERD

01 DB 설치 (My SQL - 다운로드 페이지 접속)



- 검색창에 mysql community download 입력하고 Download URL을 클릭해서 페이지로 이동합니다.

- MySQL Installer for Windows 클릭합니다.

01 DB 설치 (My SQL - 설치 파일 다운로드)

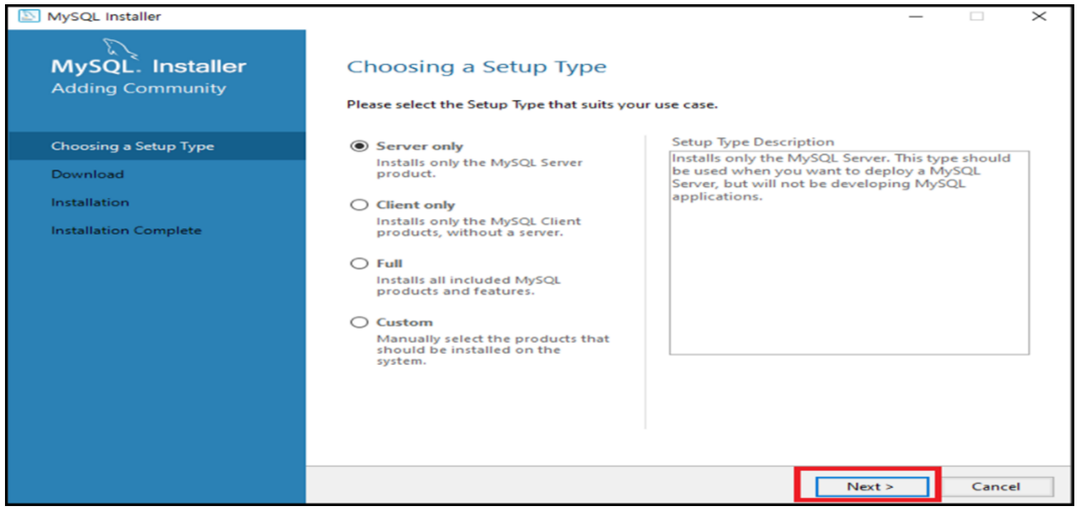


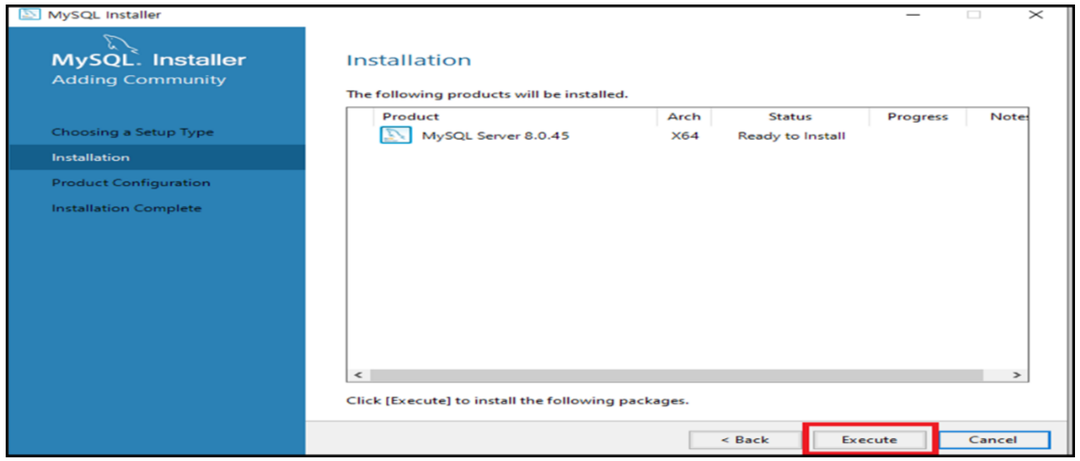


- 상단의 작은 용량(2.1M)는 온라인 설치 버전이고, 하단의 큰 용량(556.0M)은 오프라인 전체 설치 버전입니다. 하단 버전의 Download 버튼을 클릭하여 전체 설치 파일을 받습니다.

- 로그인 / 회원가입을 유도하는 페이지이지만, Oracle 계정 없이도 설치 파일은 받을 수 있으므로 "No thanks, just start my download" 링크를 클릭하면 즉시 다운로드가 시작됩니다.

01 DB 설치 (MySQL - 설치 유형 선택 및 Execute)

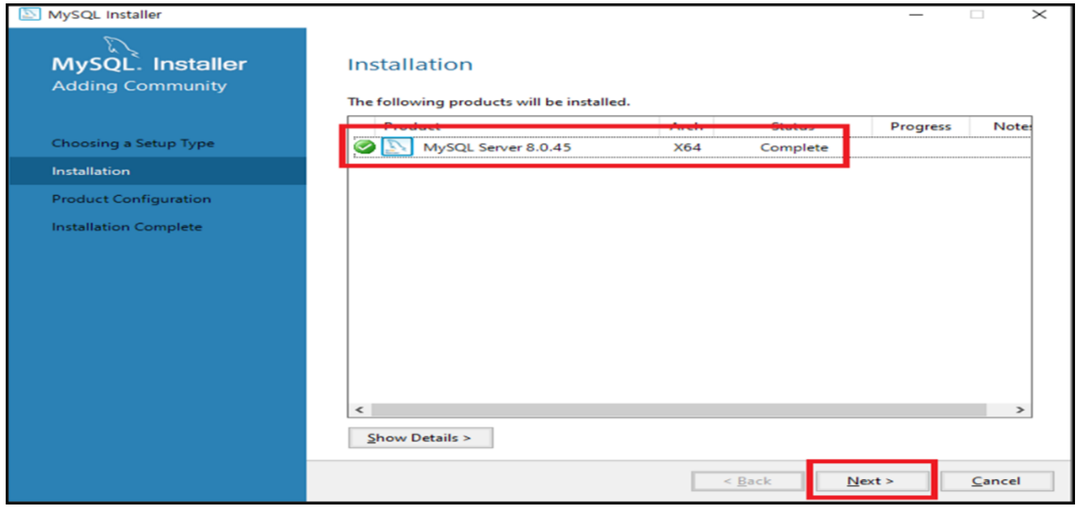


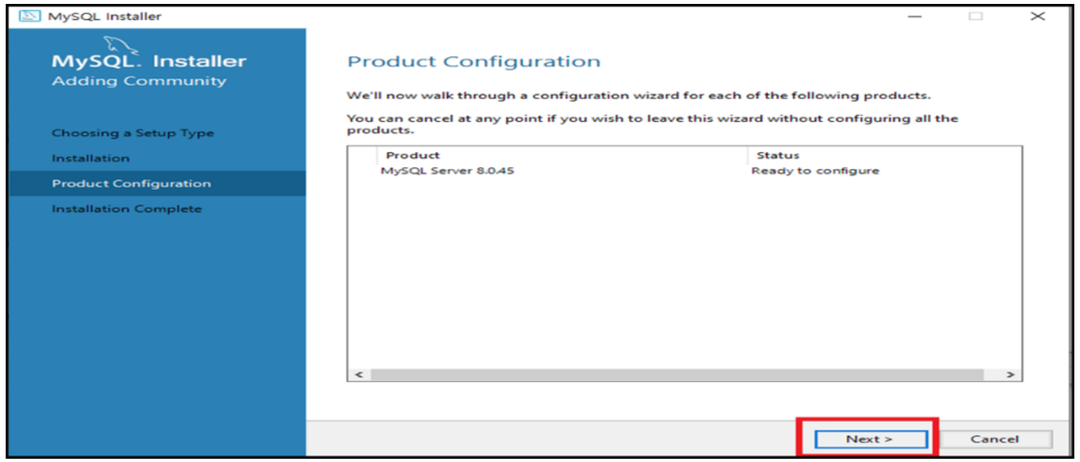


- 여러 유형 중 **"Server only"**를 선택합니다. 이는 MySQL 서버 기능만 설치하는 최소 구성으로, 데이터 서버 기능만 필요한 경우에 적합합니다.

- Execute 항목을 클릭하여 파일 설치를 시작합니다.

01 DB 설치 (MySQL - 설치 확인 및 제품 구성 진입)

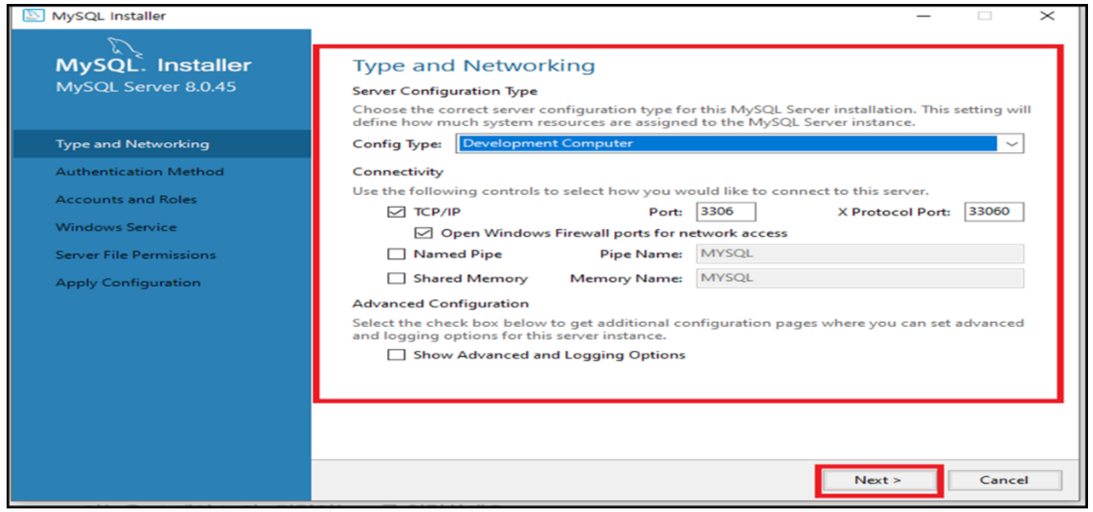


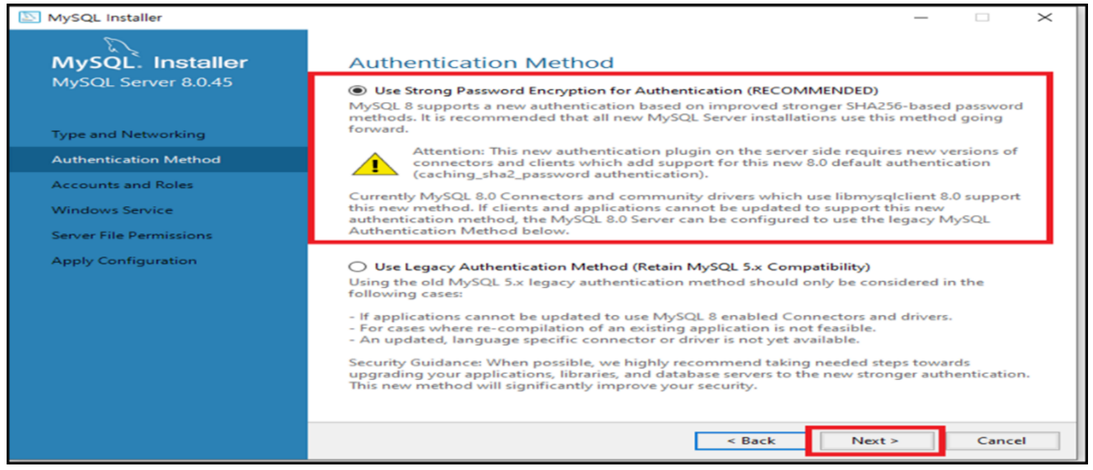


- Complete를 통해 MySQL Server 설치가 완료된 것을 확인할 수 있습니다.

- 실제 사용을 위한 환경 설정(Product Configuration) 단계로 넘어갈 차례입니다.

01 DB 설치 (My SQL - Type and Networking 설정)

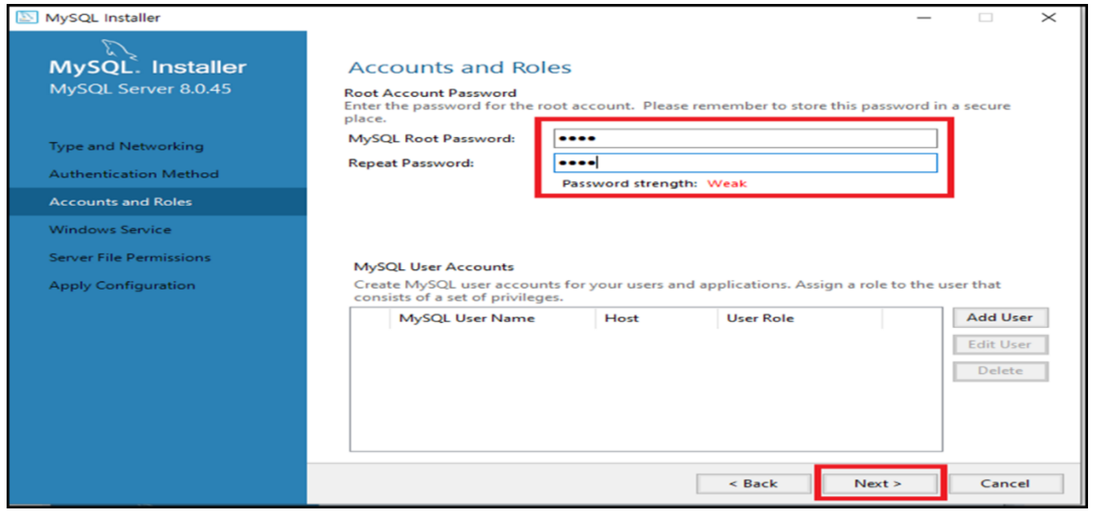




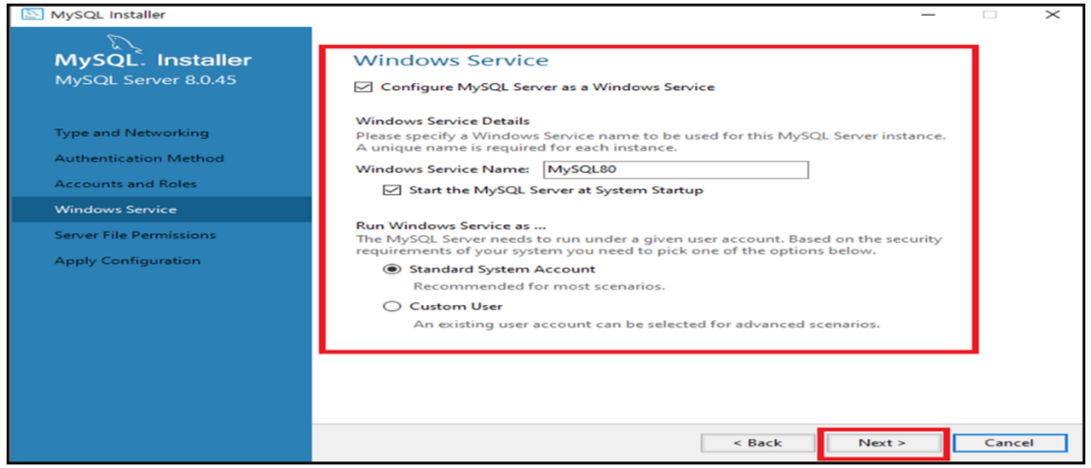
- **Config Type** : 컴퓨터의 메모리를 얼마나 점유할지를 결정하는 항목입니다. 개인 학습 환경에서는 기본 값을 사용하는 것이 일반적입니다.
- **Connectivity - TCP/IP & Port 3306** : 클라이언트가 서버에 어떤 방식으로 접속할지를 정하는 규칙입니다. 3306은 기본 통신 포트이며, 변경하지 않고 기본 값을 유지합니다.

- **Use Strong Password Encryption for Authentication** : 비밀번호를 보다 안전하게 암호화하는 최신 방식을 사용하겠다는 의미로 기본 값을 유지합니다.

01 DB 설치 (My SQL - 계정 비밀번호 설정 및 Windows Service 설정)

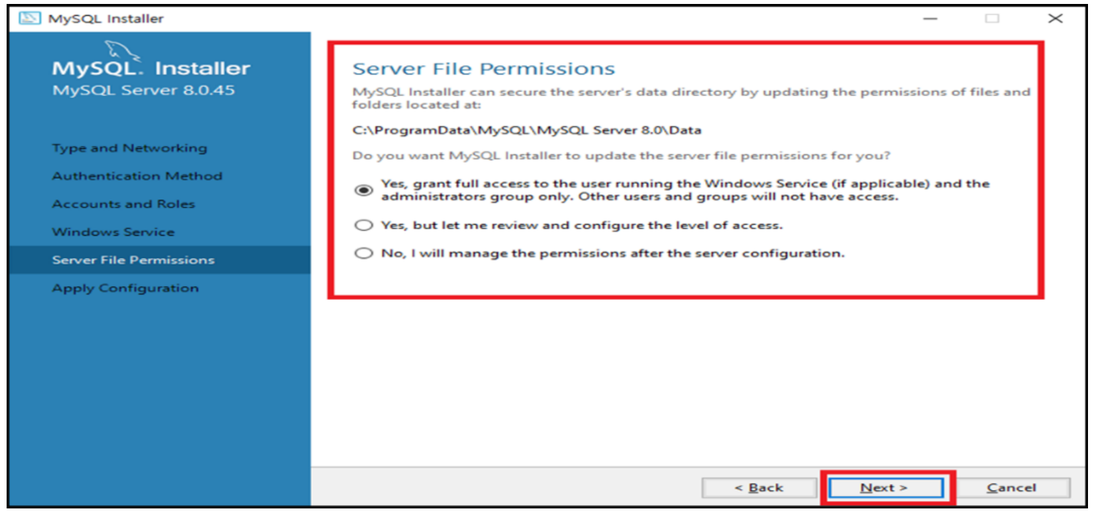


- **Accounts and Roles** : 최고 관리자 계정인 Root 계정의 비밀번호를 설정하는 단계입니다. 학원 실습 환경에서는 1234로 설정합니다.
- **MySQL User Accounts** : 일반 사용자 계정을 미리 생성할 수 있는 공간이지만 지금은 생략합니다.

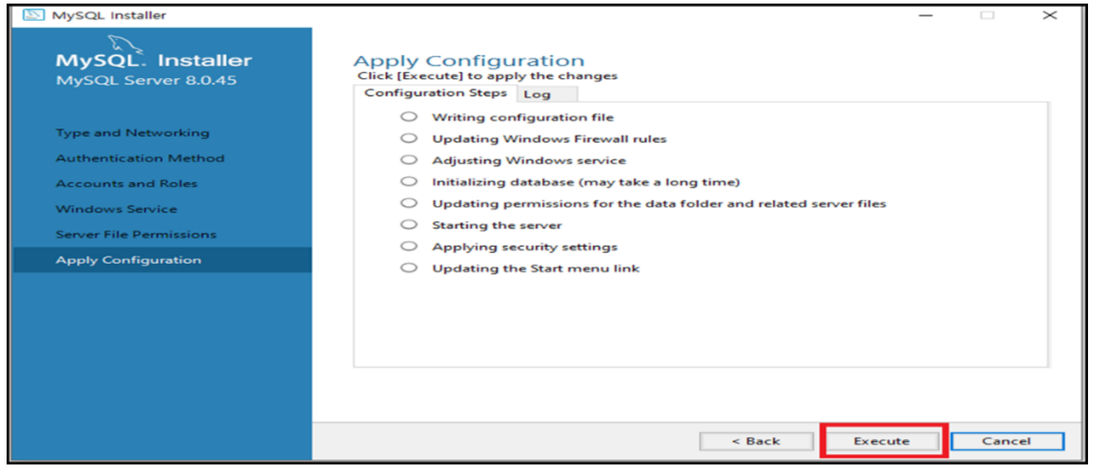


- **Windows Service** : MySQL을 Windows 서비스로 등록하는 단계입니다. 서비스로 등록하면 컴퓨터 시작 시 MySQL이 백그라운드에서 자동으로 실행됩니다. 기본값을 유지해줍니다.

01 DB 설치 (My SQL - 파일 권한 설정 및 설정값 적용)

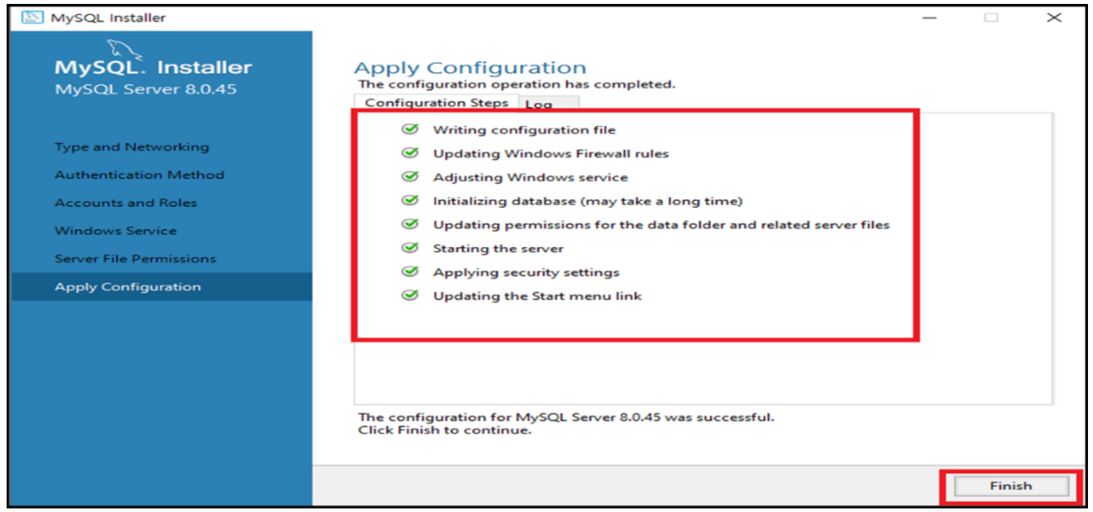


- **Server File Permissions** : 사용하는 데이터 파일에 대한 접근 권한을 설정하는 단계입니다.

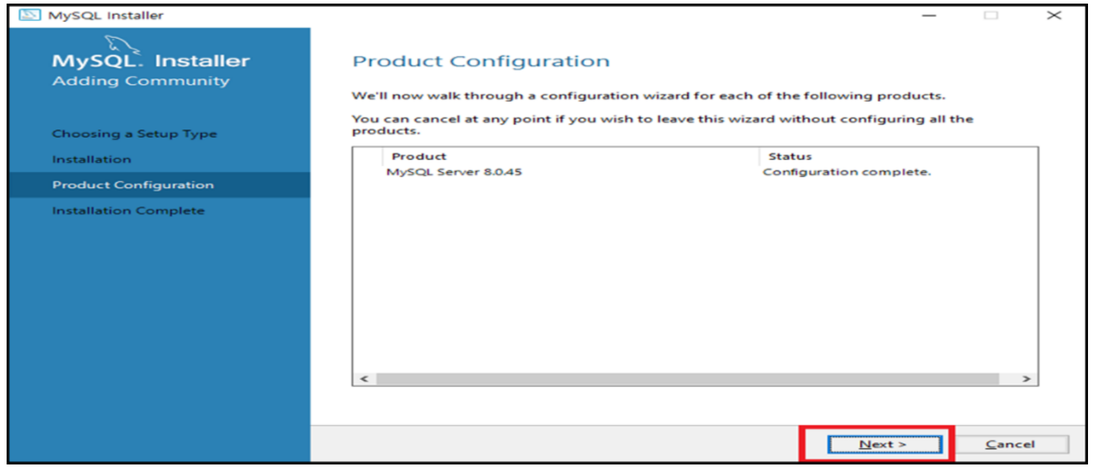


- **Apply Configuration** : 지금까지 선택한 모든 설정 값을 시스템에 실제로 적용하고, MySQL 데이터베이스 엔진을 가동하는 과정입니다.

01 DB 설치 (My SQL - 설치 완료 확인 및 마무리)

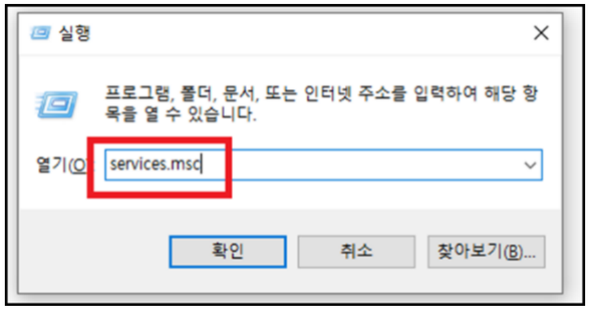


- 모든 항목에 초록색 체크 표시가 확인되면 MySQL 서버 설치와 기본 설정이 성공했다는 의미입니다.



- **Product Configuration** : MySQL Server에 대한 세부 설정이 모두 끝났음을 확인해주는 '제품 구성' 요약 페이지 입니다.

01 DB 설치 (My SQL - 서비스 동작 확인)







- Win + R 단축키로 실행창을 열고 service.msc를 입력하여 Windows 서비스 관리자를 실행합니다.
- 서비스 목록에서 MySQL 항목을 찾아 "실행 중" 상태인지 확인합니다.

- 시작 메뉴 검색창에 **"MySQL"** 을 입력하여 MySQL Command Line Client를 실행 할 수 있는지 확인 합니다.

01 DB 설치 (My SQL - MySQL 접속 확인)

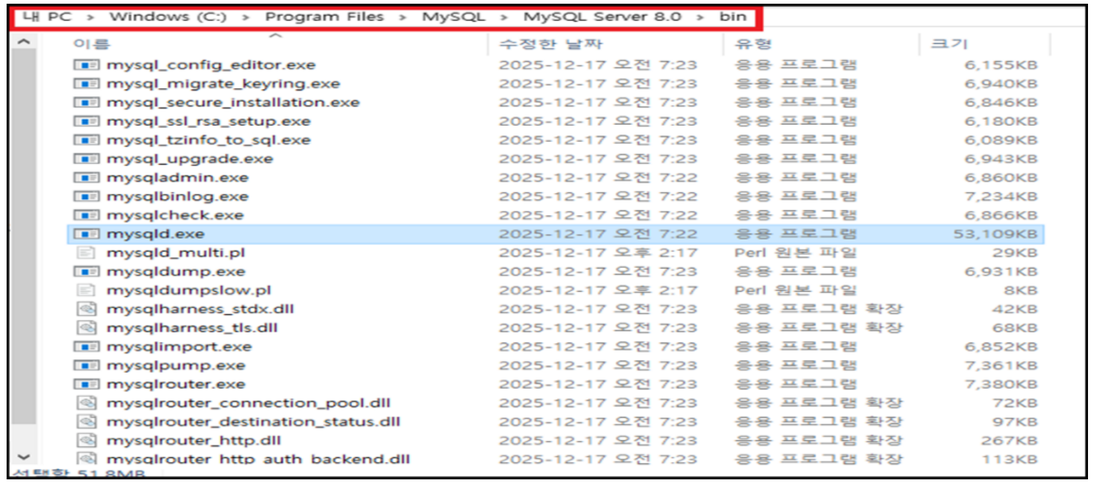


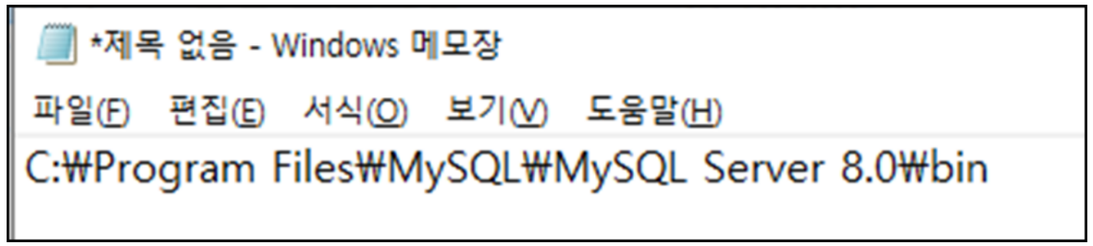
- MySQL Command Line Client 가 실행되면 비밀번호 입력창이 나타납니다. 앞서 설정한 패스워드 1234를 입력하면 MySQL 서버에 접속이 됩니다.



- 명령어 입력 프롬포트에 **mysql>** 가 표시되면 MySQL이 정상적으로 동작하고 있음을 의미합니다.

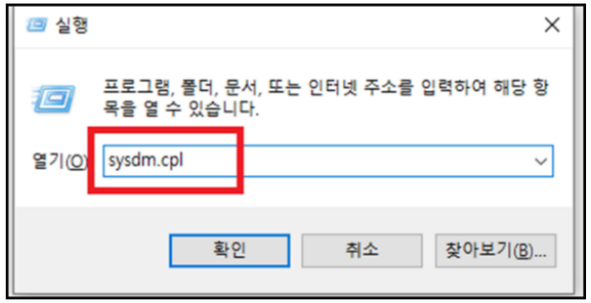
01 DB 설치 (My SQL - 설치 경로 복사 및 저장)

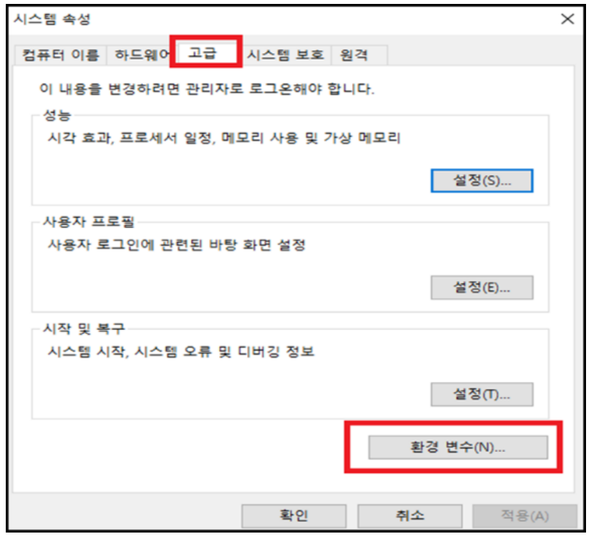




- MySQL이 설치된 경로(디렉토리 주소)를 복사합니다. 이 경로는 환경 변수를 등록할 때 사용되므로, 메모장을 실행 후 해당 내용을 붙여넣기 하여 저장해 둡니다. 일반적으로 설치 형태는 C:\ProgramFiles\MySQL\MySQL Server 8.0\bin 입니다.

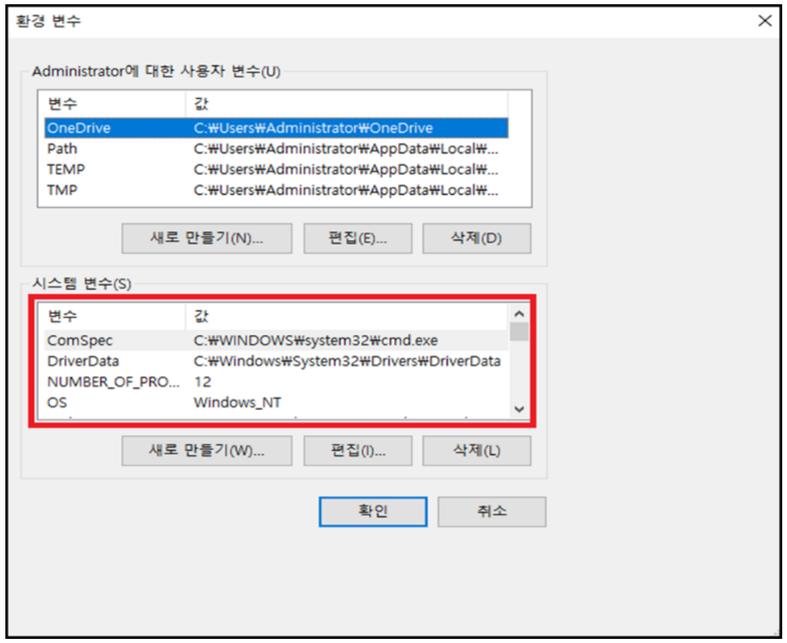
01 DB 설치 (환경변수 - 등록)

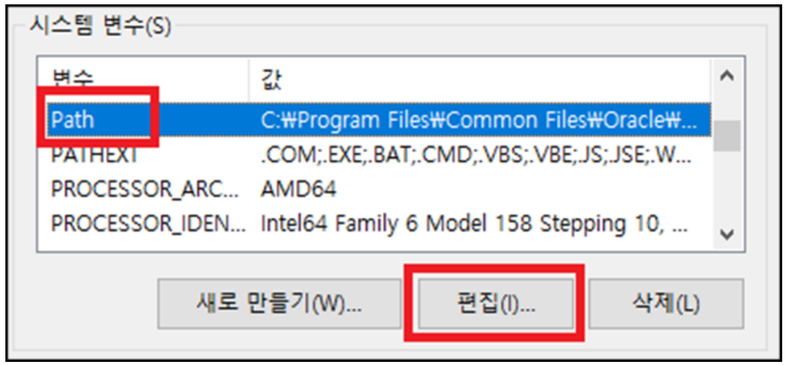




- Win + R 단축키로 실행창을 열고 "**sysdm.cpl**"을 입력하여 시스템 속성 창을 열어주고 고급 탭에서 환경 변수를 클릭합니다.

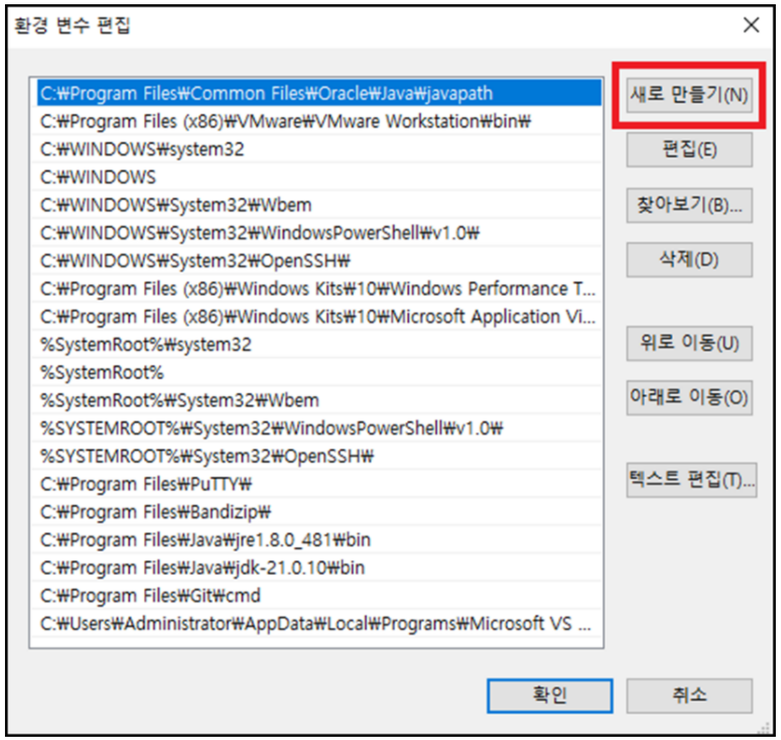
01 DB 설치 (환경변수 - 등록)

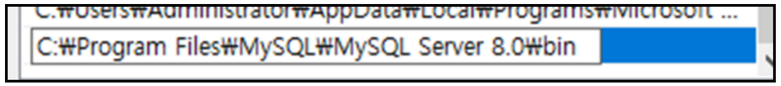




- 환경 변수 창 하단의 **"시스템 변수"** 영역에서 임의의 변수를 클릭한 뒤 키보드에서 "P" 키를 누르면 P로 시작하는 변수로 빠르게 이동 할 수 있습니다. 이 방법으로 **"Path"** 변수를 찾은 후 선택하고 **"편집"** 버튼을 클릭합니다.
- Path 변수는 운영체제가 명령어를 실행할 때 참조하는 디렉토리 목록으로, 여기에 MySQL 경로를 추가하면 CMD에서 mysql 명령어를 인식할 수 있게 됩니다.

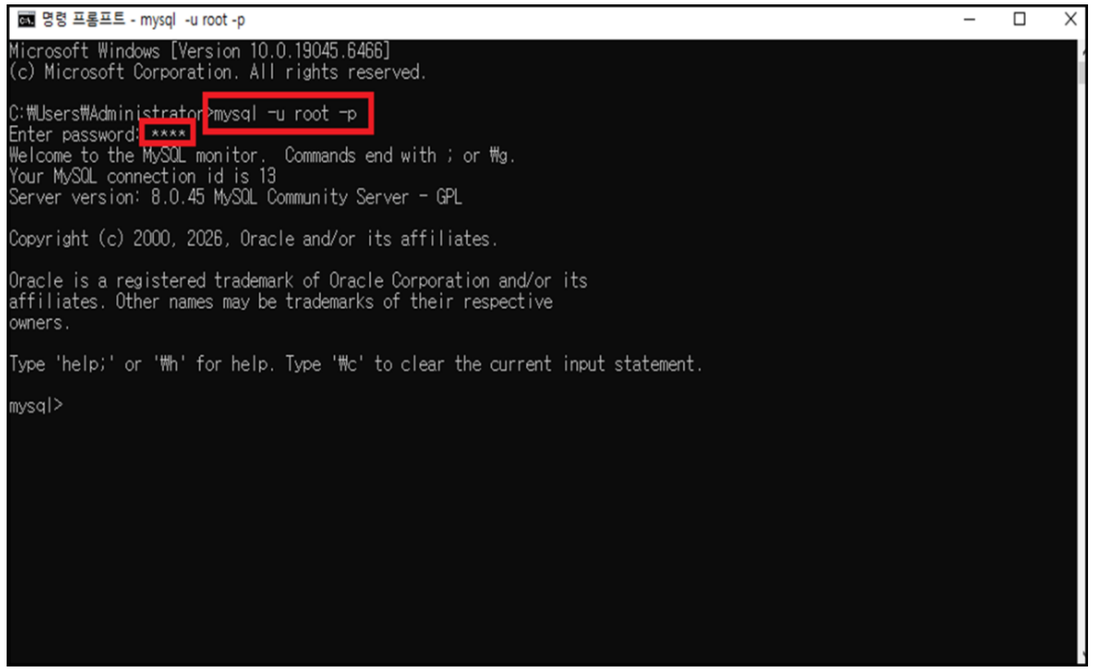
01 DB 설치 (환경변수 - 등록)





- Path 편집 창이 열리면 우측의 "새로 만들기" 버튼을 클릭합니다. 이전에 메모장에 저장 해둔 MySQL 설치 경로 **(C:\Program Files\MySQL\MySQL Server 8.0\bin)** 를 붙여넣기한 뒤 확인을 눌러 저장합니다. 이로써 CMD 어느 위치에서든 mysql 명령어를 사용할 수 있는 환경이 만들어집니다.

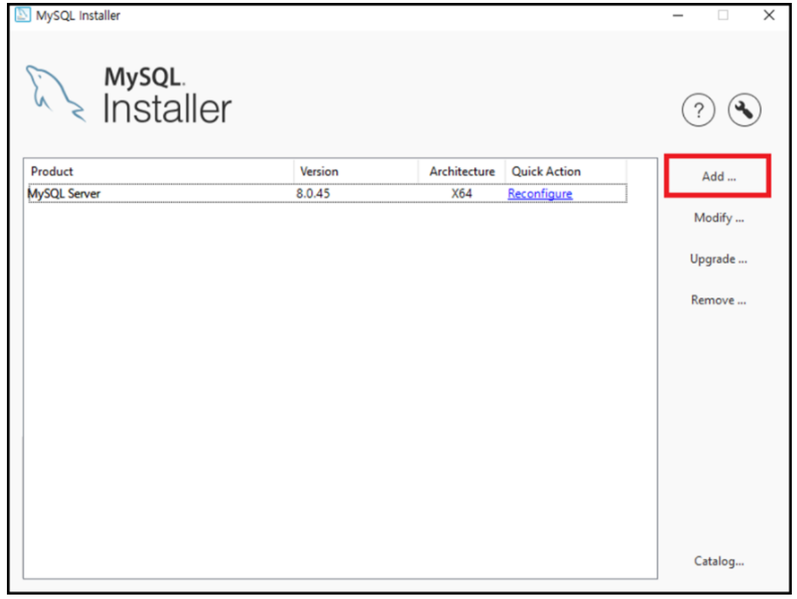
01 DB 설치 (환경변수 - CMD에서 MySQL 접속 확인)



- 명령 프롬포트 (CMD) 를 열고 mysql -u root -p 명령어를 입력하여 MySQL 접속 테스트를 합니다. 이후 패스워드 입력 프롬포트에 1234를 입력하면 MySQL 서버에 정상적으로 접속됩니다. 이 단계에서 정상 접속이 확인되면 환경 변수 등록이 올바르게 완료된 것이며, 이후부터는 CMD에서 언제든지 MySQL에 접속할 수 있습니다.

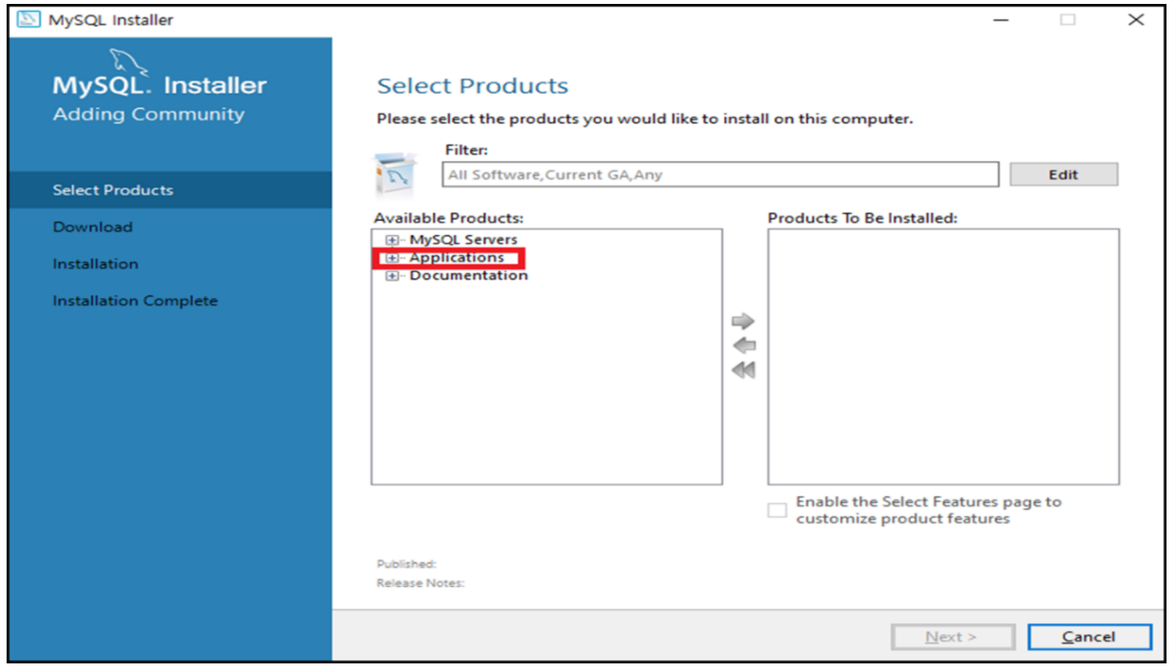
01 DB 설치 (워크벤치 - MySQL Installer에서 Workbench 추가)

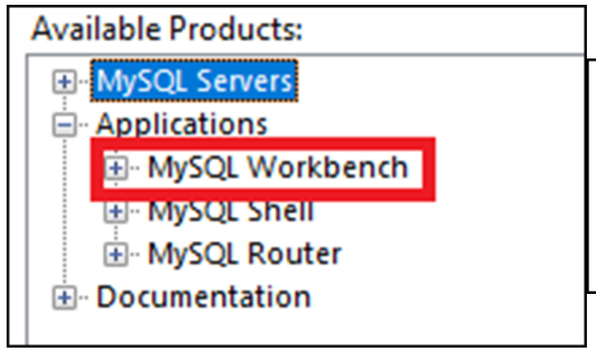


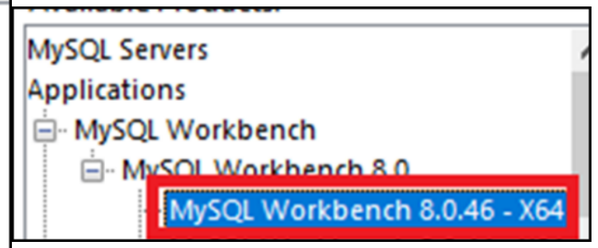


- 시작메뉴에서 MySQL Installer 입력 후 실행합니다. Workbench는 MySQL 데이터베이스를 GUI(그래픽 인터페이스) 환경에서 관리할 수 있는 공식 도구입니다. **"Add"** 버튼을 클릭하여 다음 화면으로 이동합니다.

01 DB 설치 (워크벤치 - MySQL Workbench 선택)

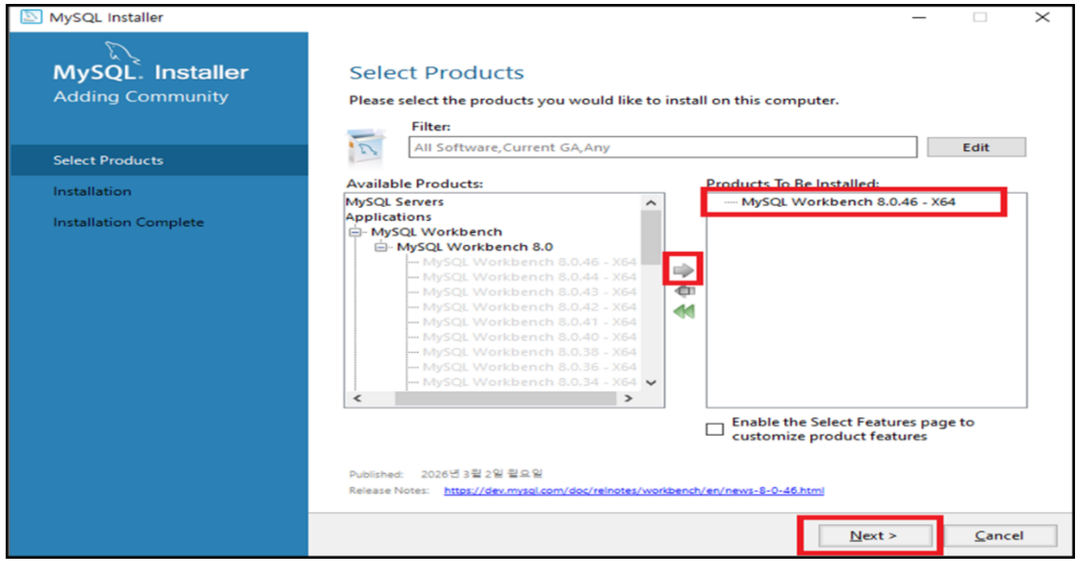


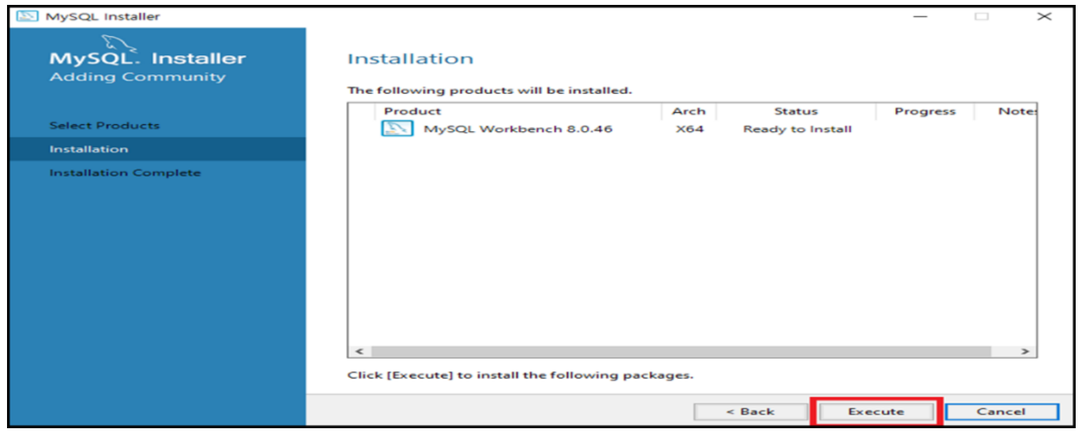




- Application → MySQLWorkbench → MySQL Workbench 8.0.46 - X64순서로 선택을 진행합니다. X64는 64비트 운영체제용 버전을 의미합니다.

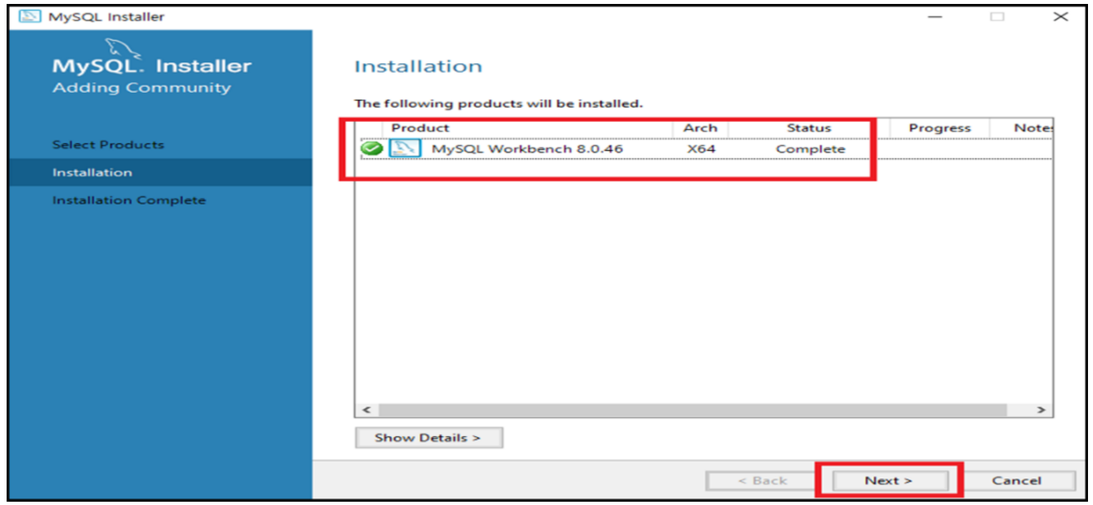
01 DB 설치 (워크벤치 설정)

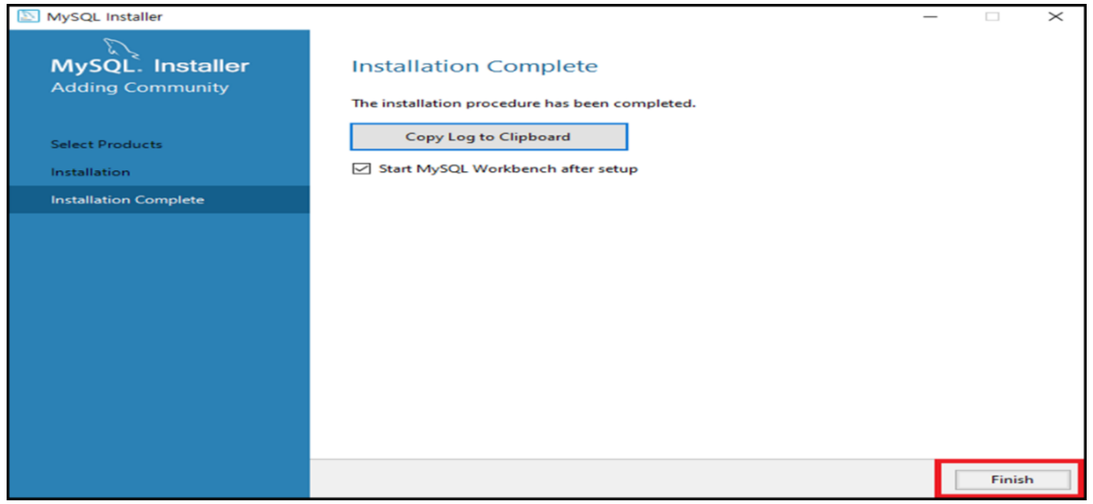




- 선택한 Workbench 항목 옆의 화살표(→)를 클릭하여 **"Products To Be Installed"** 목록에 추가되었는지 확인 한 뒤 Next를 클릭합니다.
- 이어서 Execute 버튼을 클릭하면 Workbench 설치가 시작됩니다.

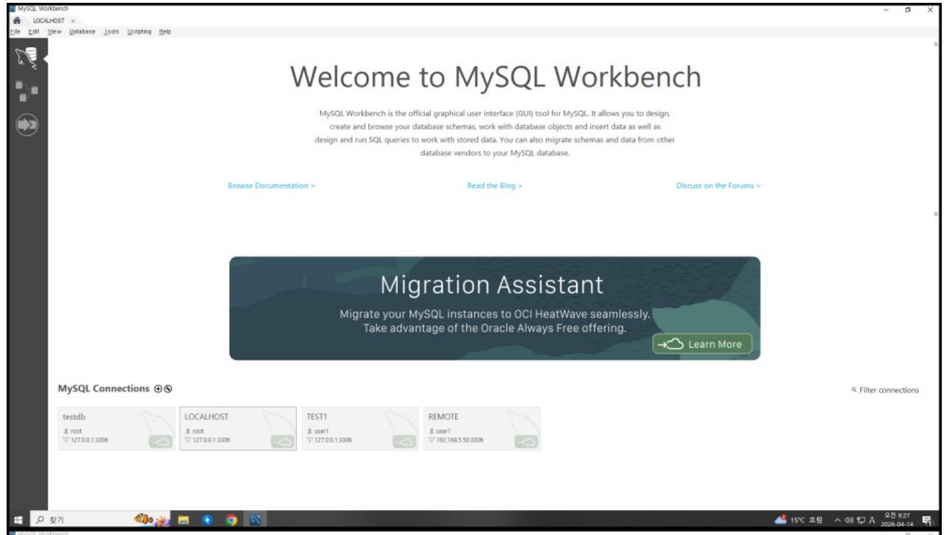
01 DB 설치 (워크벤치 설정)

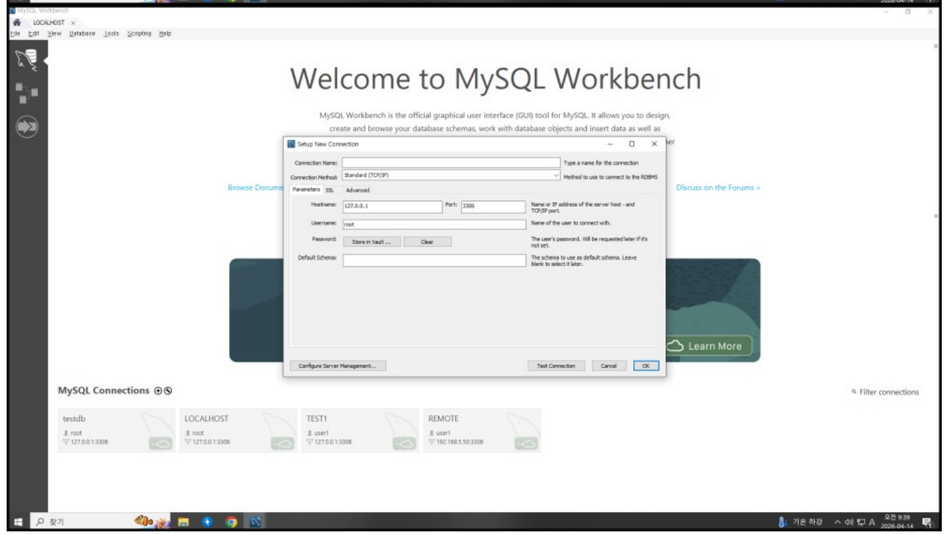


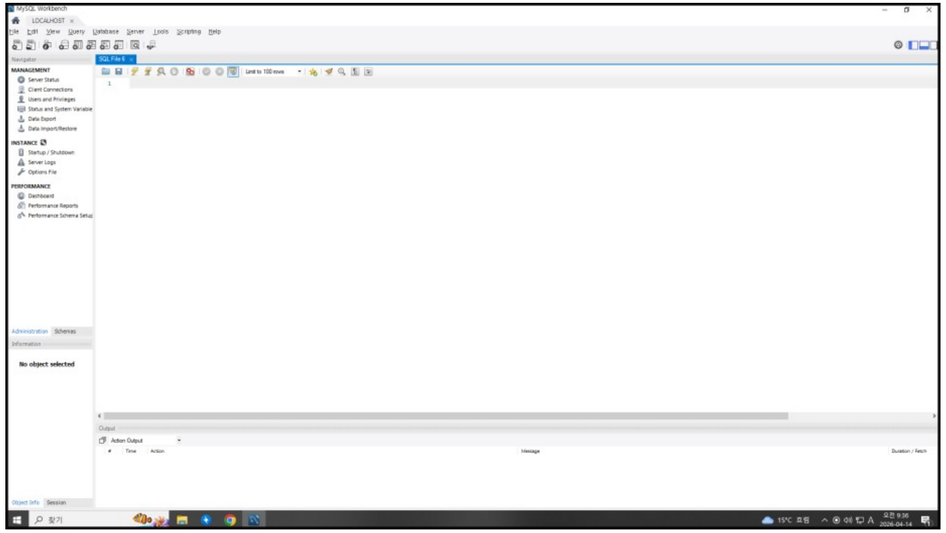


- 설치가 완료되면 Installation 목록에서 Workbench가 정상적으로 설치되었는지 확인합니다. 이상이 없으면 Next를 클릭하고, 최종적으로 Finish 버튼을 눌러 MySQL Workbench 설치를 마무리 합니다. 이후 Workbench를 실행하여 MySQL서버에 GUI로 접속할 수 있습니다.

01 DB 설치 (워크벤치 실행)

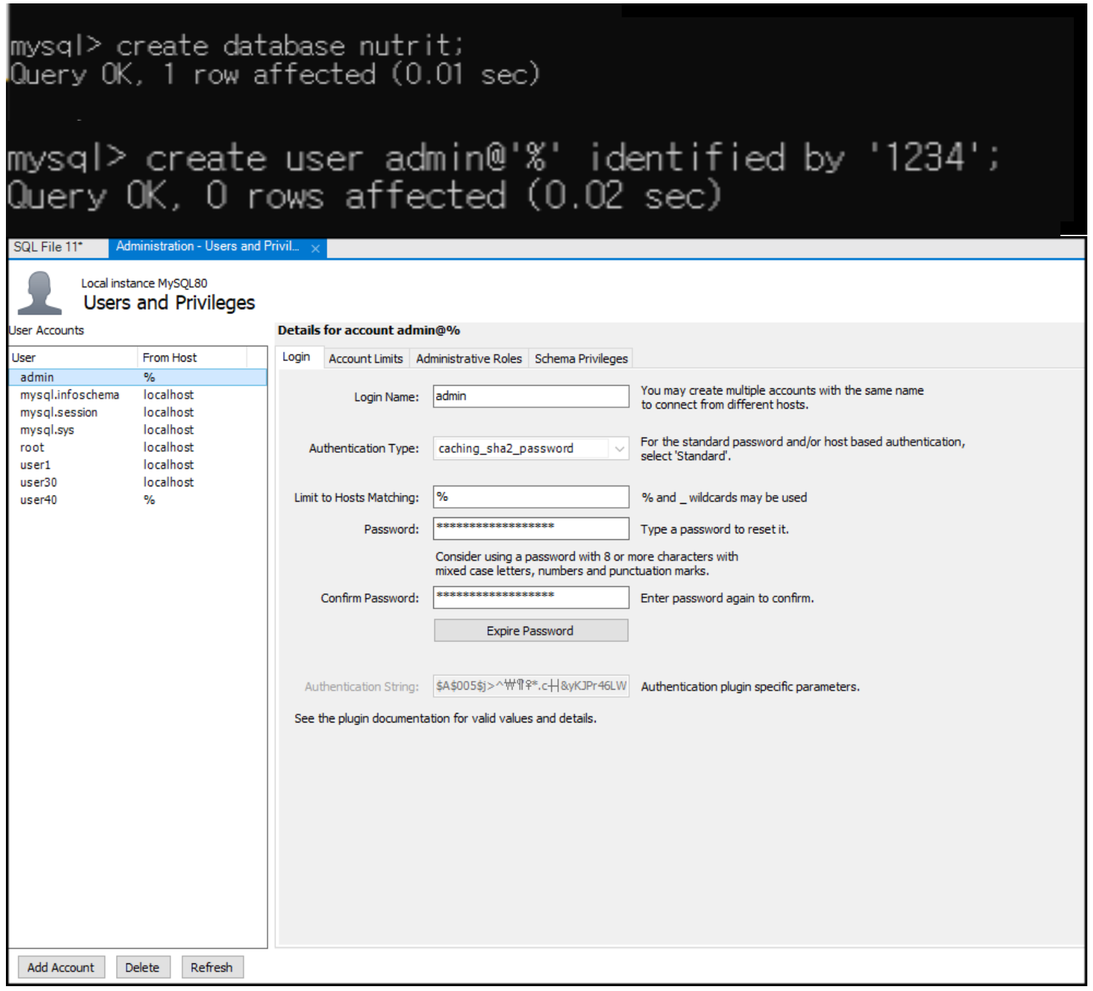






- MySQL Workbench 8.0을 눌러 실행해보고 정상 작동하는지 확인합니다.

02 DB 계정생성



- nutrit db를 생성합니다.
- 원격접속 가능하도록 '%' 권한으로 'admin' 계정을 생성합니다. (비밀번호 1234)
- mysql administration 에서 계정 권한 정보를 확인하고 계정이 추가되었는지 확인합니다.

02 DB 계정생성

```
mysql> grant all privileges on nutrit.* to admin@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)

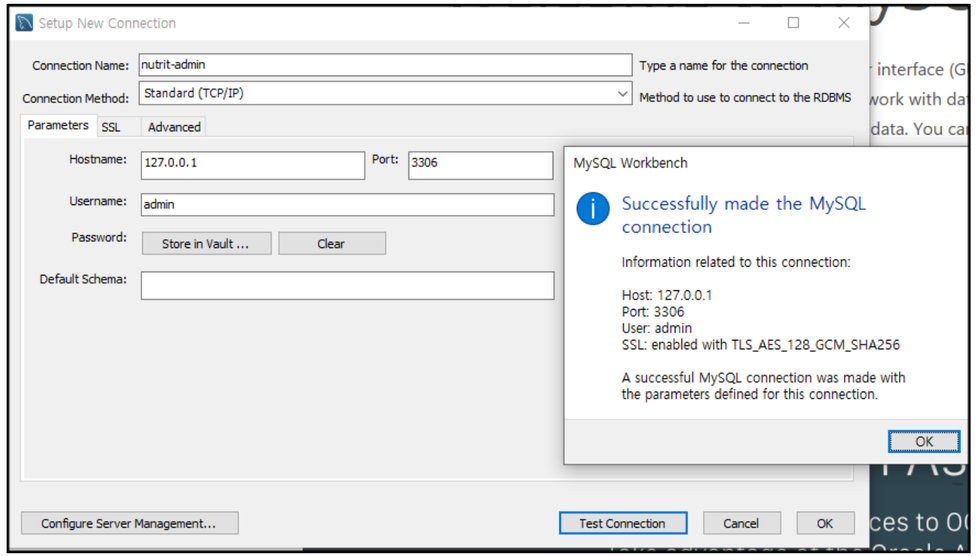
mysql> show grants for admin@'%';
+---------------------------------------------------+
| Grants for admin@%                                |
+---------------------------------------------------+
| GRANT USAGE ON *.* TO `admin`@`%`                 |
| GRANT ALL PRIVILEGES ON `nutrit`.* TO `admin`@`%` |
+---------------------------------------------------+
2 rows in set (0.00 sec)
```

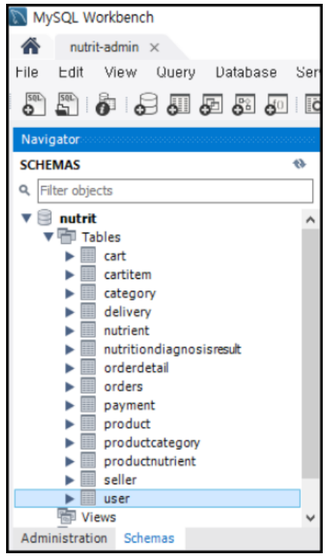
- grant all privileges on nutrit.* to admin@'%'; 을 사용하여 'nutrit db 모든 권한' 을 admin에게 부여합니다.
- show grants for admin@'%' 을 사용하여 확인 작업을 진행합니다.
- Workbench에서도 권한이 잘 부여되어 있는지 확인 작업을 진행합니다.



Schema	Privileges
nutrit	ALTER, ALTER ROUTINE, CREATE, CREATE ROUTINE, CREATE TEMPORARY TABLES, CREATE VIEW, DELETE, DROP, EVENT, EXECUTE, INDEX, INSERT, LOCK TABLES, REFERENCES, SELECT, SHOW VIEW, TRIGGER, UPDATE

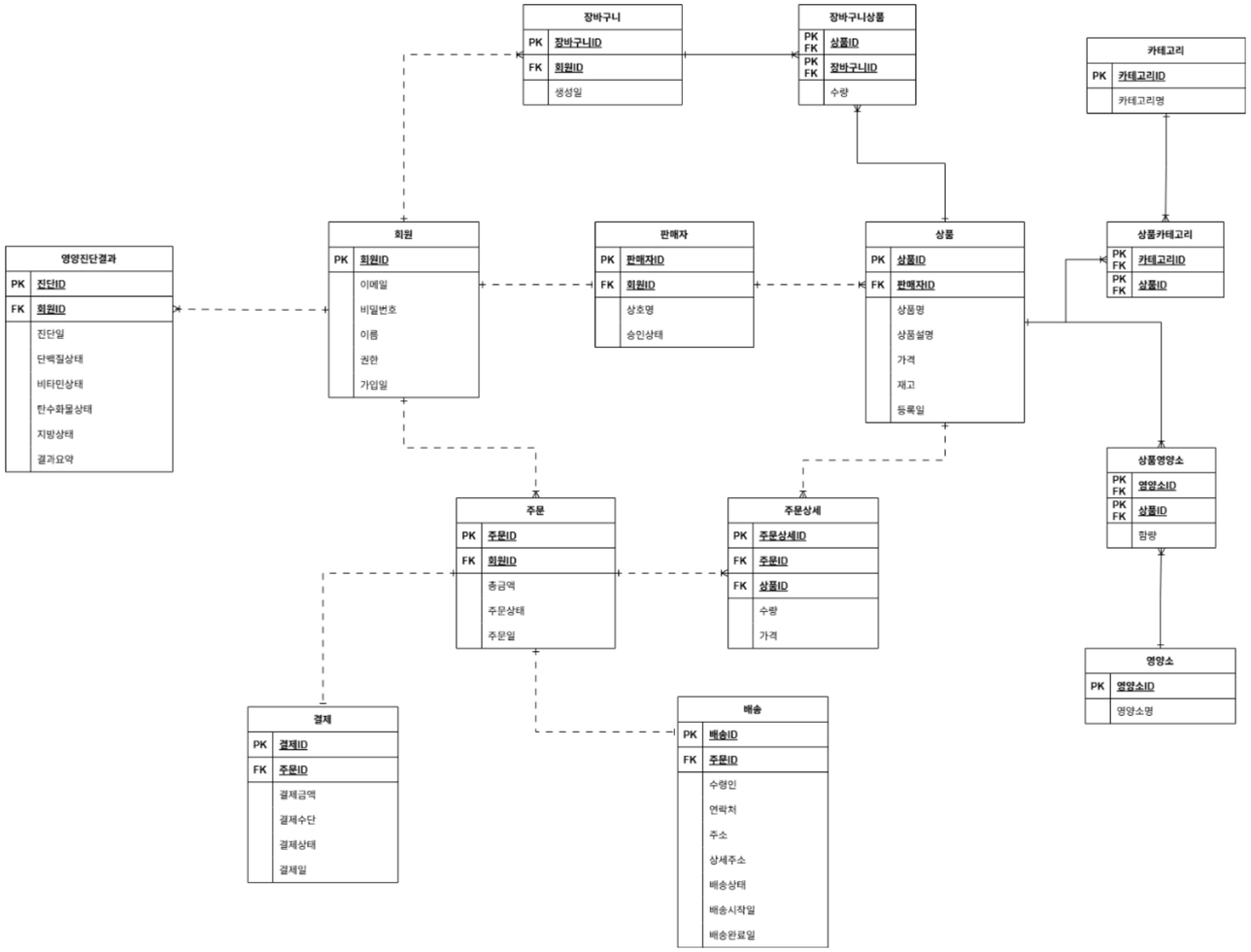
02 DB 계정생성



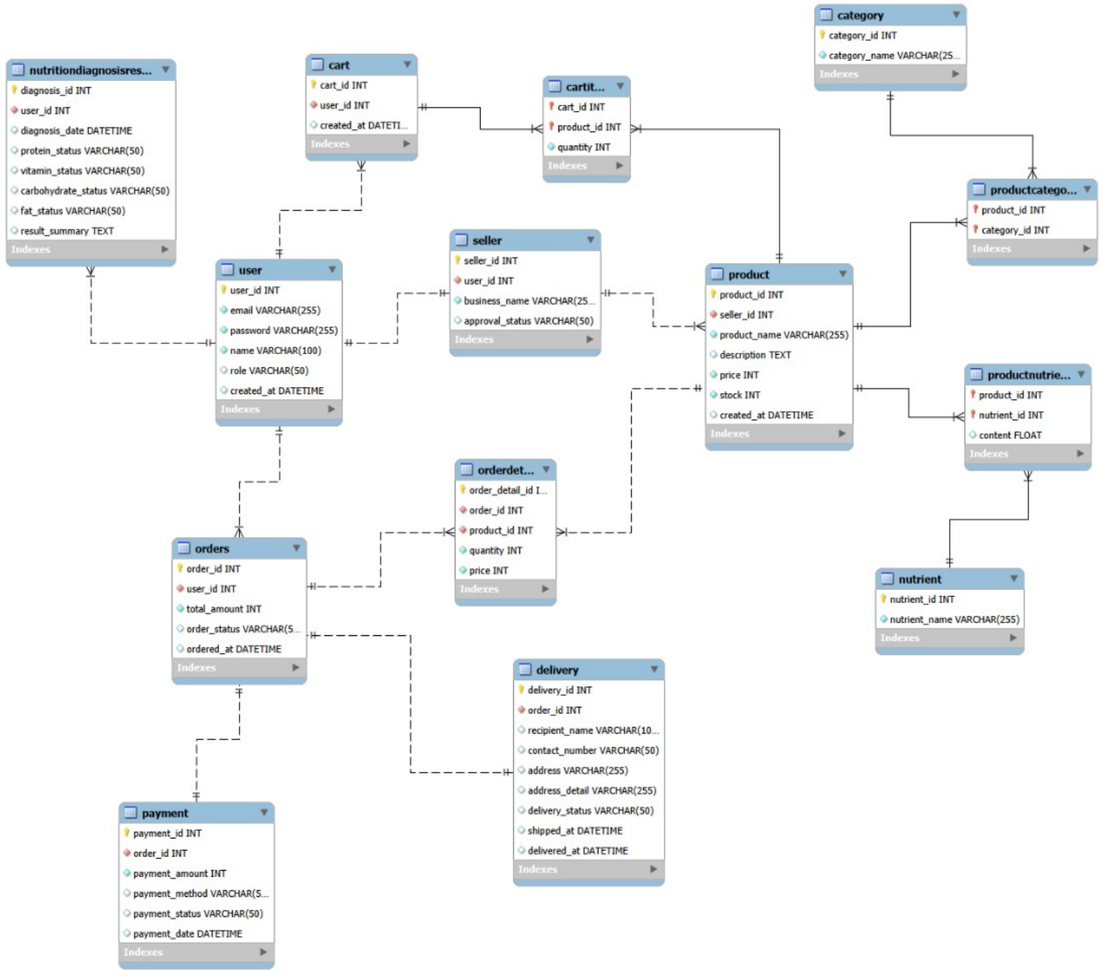


- 'admin' 계정으로 원격접속을 진행하여 확인합니다.
- 생성한 'nutrit' db 권한 여부 및 테이블 상태를 확인합니다.

03 DB ERD (draw.io)



03 DB ERD (MY SQL)



03 DB ERD (SQL)

```
-- User Table
CREATE TABLE User (
user_id INT AUTO_INCREMENT PRIMARY KEY,
email VARCHAR(255) NOT NULL UNIQUE,
password VARCHAR(255) NOT NULL,
name VARCHAR(100) NOT NULL,
role VARCHAR(50),
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Seller Table (1:1 with User)
CREATE TABLE Seller (
seller_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL UNIQUE,
business_name VARCHAR(255) NOT NULL,
approval_status VARCHAR(50),
CONSTRAINT fk_seller_user
FOREIGN KEY (user_id)
REFERENCES User(user_id)
);
```

```
-- Product Table
CREATE TABLE Product (
product_id INT AUTO_INCREMENT PRIMARY KEY,
seller_id INT NOT NULL,
product_name VARCHAR(255) NOT NULL,
description TEXT,
price INT NOT NULL,
stock INT NOT NULL,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_product_seller
FOREIGN KEY (seller_id)
REFERENCES Seller(seller_id)
);

-- Category Table
CREATE TABLE Category (
category_id INT AUTO_INCREMENT PRIMARY KEY,
category_name VARCHAR(255) NOT NULL
);
```

```
-- Product-Category Mapping Table (M:N)
CREATE TABLE ProductCategory (
product_id INT,
category_id INT,
PRIMARY KEY (product_id, category_id),
CONSTRAINT fk_productcategory_product
FOREIGN KEY (product_id)
REFERENCES Product(product_id),
CONSTRAINT fk_productcategory_category
FOREIGN KEY (category_id)
REFERENCES Category(category_id)
);
```

03 DB ERD (SQL)

```
-- Orders Table
CREATE TABLE Orders (
order_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
total_amount INT NOT NULL,
order_status VARCHAR(50),
ordered_at DATETIME DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_orders_user
FOREIGN KEY (user_id)
REFERENCES User(user_id)
);
```

```
-- Order Detail Table
CREATE TABLE OrderDetail (
order_detail_id INT AUTO_INCREMENT PRIMARY KEY,
order_id INT NOT NULL,
product_id INT NOT NULL,
quantity INT NOT NULL,
price INT NOT NULL,
CONSTRAINT fk_orderdetail_order
FOREIGN KEY (order_id)
REFERENCES Orders(order_id),
CONSTRAINT fk_orderdetail_product
FOREIGN KEY (product_id)
REFERENCES Product(product_id)
);
```

```
-- Nutrition Diagnosis Result Table
CREATE TABLE NutritionDiagnosisResult (
diagnosis_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL,
diagnosis_date DATETIME DEFAULT CURRENT_TIMESTAMP,
protein_status VARCHAR(50),
vitamin_status VARCHAR(50),
carbohydrate_status VARCHAR(50),
fat_status VARCHAR(50),
result_summary TEXT,
CONSTRAINT fk_diagnosis_user
FOREIGN KEY (user_id)
REFERENCES User(user_id)
);

-- Nutrient Table
CREATE TABLE Nutrient (
nutrient_id INT AUTO_INCREMENT PRIMARY KEY,
nutrient_name VARCHAR(255) NOT NULL
);
```

03 DB ERD (SQL)

```
-- Product-Nutrient Mapping Table (M:N)
CREATE TABLE ProductNutrient (
product_id INT,
nutrient_id INT,
content FLOAT,
PRIMARY KEY (product_id, nutrient_id),
CONSTRAINT fk_productnutrient_product
FOREIGN KEY (product_id)
REFERENCES Product(product_id),
CONSTRAINT fk_productnutrient_nutrient
FOREIGN KEY (nutrient_id)
REFERENCES Nutrient(nutrient_id)
);
```

```
-- Payment Table (1:1 with Orders)
CREATE TABLE Payment (
payment_id INT AUTO_INCREMENT PRIMARY KEY,
order_id INT NOT NULL UNIQUE,
payment_amount INT NOT NULL,
payment_method VARCHAR(50),
payment_status VARCHAR(50),
payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_payment_order
FOREIGN KEY (order_id)
REFERENCES Orders(order_id)
);
```

```
-- Cart Table (1:1 with User)
CREATE TABLE Cart (
cart_id INT AUTO_INCREMENT PRIMARY KEY,
user_id INT NOT NULL UNIQUE,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_cart_user
FOREIGN KEY (user_id)
REFERENCES User(user_id)
);
-- Cart Item Table
CREATE TABLE CartItem (
cart_id INT,
product_id INT,
quantity INT NOT NULL,
PRIMARY KEY (cart_id, product_id),
CONSTRAINT fk_cartitem_cart
FOREIGN KEY (cart_id)
REFERENCES Cart(cart_id),
CONSTRAINT fk_cartitem_product
FOREIGN KEY (product_id)
REFERENCES Product(product_id)
);
```

03 DB ERD (SQL)

```
-- Delivery Table (1:1 with Orders)
CREATE TABLE Delivery (
delivery_id INT AUTO_INCREMENT PRIMARY KEY,
order_id INT NOT NULL UNIQUE,
recipient_name VARCHAR(100),
contact_number VARCHAR(50),
address VARCHAR(255),
address_detail VARCHAR(255),
delivery_status VARCHAR(50),
shipped_at DATETIME,
delivered_at DATETIME,
CONSTRAINT fk_delivery_order
FOREIGN KEY (order_id)
REFERENCES Orders(order_id)
);
```